Ders 14 Çalışma Özeti

Ders 14 Çalışma Özeti

Genel Konular

- Büyük dil modelleri (LLM) (ekrandan, 00:20:00; slaytlar “bolum8_LLM”, 27 slayt)
  - Dersin planı: otoregresif dil modelleri, uyarlanmış (fine-tune edilmiş) büyük dil modelleri, bu işlerde kullanılan yöntemler ve bir tartışma slaytı.
- Dil modelleri nasıl eğitilir (ekrandan, 00:20:00)
  - Eğiticili (supervised) öğrenmede girdi-çıktı eşlemesi gerekir. Çıktıyı insan gücüyle hazırlamak pahalıdır ve eğitim kümesinin büyüklüğünü sınırlar.
  - Daha ucuz yol: orijinal metni zaten biliyoruz; metni bozup modelden orijinalini tahmin etmesini isteriz. Üç yaygın biçim: next (sonrakini tahmin), masked (maskelenen parçayı tahmin), changed (değiştirilen parçayı bulma; değişiklik kurallarla ya da başka bir modelle yapılır).
  - Slayta göre artık otoregresif modeller (next thing prediction) standarttır, çünkü yeni görevlere çok daha kolay uyum sağlarlar. “Thing” kelime, token, harf ya da cümle olabilir. Önceden ağırlıklı olarak maskeli modelle çalışan firmalar bile next token prediction’a geçmiştir.
  - BERT maskeli dil modellerine ilgiyi artırdı. GPT’nin Transformer’lı hali BERT’ten sonra çıktı, ama next token prediction yeni bir fikir değildir. Hoca, yanlış hatırlamıyorsa geri yayılımın (backpropagation) önerildiği makalede bile küçük bir sonraki kelime tahmini deneyi olduğunu söyledi.
  - Popülerlik Kasım-Aralık 2022’de ChatGPT’nin kamuya açık gösterimiyle başladı.
- LLM’ler eğitim kümesinde sıfır hataya erişebilir mi? (ekrandan, 00:30:00)
  - Küçük veri ve büyük modelle ezberleme teknik olarak mümkündür. Ama dilde aynı başlangıçla başlayan çok sayıda cümle farklı biçimlerde devam eder (“LLM’ler eğitim kümesinde …” diye başlayan cümleler gibi).
  - Klasik makine öğrenmesi diliyle: aynı noktada farklı sınıflardan örnekler vardır. Bu yüzden yeterince büyük bir dil derleminde sıfır hata mümkün değildir.
  - Hatayı azaltmak için model, aynı girdiden sonra gelebilecek farklı çıktıların iç temsillerini birbirine yaklaştırmak zorundadır. Slayttaki soru: düşük hata için benzer anlama/kullanıma sahip kelimelerin temsilleri benzeşir mi?
  - Dil modelinin yaptığı iş regresyon değil sınıflandırmadır: sözlük 128 bin token ise softmax ile 128 bin sınıftan biri seçilir.
- Amaçlanmadan ortaya çıkan kabiliyetler (emergent properties) (ekrandan, 00:32:10)
  - Kaynak: “Language Models are Few-Shot Learners” (GPT-3 makalesi, OpenAI).
  - Önceden eğitilmiş modellerin asıl amacı bir problemi çözmek değil, başka problemler için sağlam bir altyapı olmaktır. Ama model ve veri belli bir büyüklüğü geçince, model hiç güncellenmeden (fine-tune yapılmadan) yeni görevlerde sonuç alınabiliyor.
- Fine-tune ve X-shot (ekrandan, 00:40:00)
  - Geleneksel fine-tuning (GPT-3’te kullanılmadı): yeni görevin girdi-çıktı örnekleriyle eğitime devam edilir, her örnekte gradyan güncellemesi yapılır ve ağırlıklar değişir. Ön eğitime göre çok daha kısa ve az veriyle yapılır.
  - Cümlenin son kelimesini tahmin etme gibi bir görev için next token prediction ile eğitilmiş modeli yeniden fine-tune etmenin anlamı yoktur; görev, ön eğitim göreviyle aynıdır.
  - Zero-shot: yalnız görev tanımı ve istem verilir (“Translate English to French: cheese =>”).
  - One-shot: görev tanımına ek olarak bir örnek (“sea otter => loutre de mer”).

  - Few-shot: birkaç örnek (“peppermint => menthe poivrée”, “plush girafe => girafe peluche”).
  - Üçünde de gradyan güncellemesi yoktur. Hepsi next token prediction ile eğitilmiş modele doğrudan girdi olarak verilir. Görev tanımıyla ayrıca eğitilmiş bir şey yoktur.
- LAMBADA ve GPT-3 sonuçları (ekrandan, 00:40:00; 00:42:30)
  - LAMBADA: cümlenin son kelimesini tahmin; uzun bağımlılıkları bulma kabiliyeti.
  - X ekseni milyar cinsinden parametre sayısıdır (0.1B’den 175B’ye; 175B GPT-3’ün kendisi, ötekiler aynı veriyle eğitilmiş küçük sürümleri). 0.1B modelin zero-shot doğruluğu %40 civarındadır.
  - Parametre arttıkça üç ayarın da başarısı artar. Few-shot (K=15) büyük modellerde ötekileri aşar. One-shot bazen zero-shot’tan kötüdür; sistematik bir düzen yoktur.
  - Başka görevler: aritmetik (iki-beş basamaklı toplama/çıkarma, iki basamaklı çarpma), çeviri (Multi-BLEU), kelime karıştırma (wordscramble), insanların modelin yazdığı haberi ayırt etmesi.
  - İnsanların “bunu insan mı yazdı, makine mi” ayırt etme doğruluğu küçük modellerde %75 civarındayken model büyüdükçe %50’ye (yazı tura) indi.
  - Çeviride fine-tune edilmeden, yalnızca akıllıca tasarlanmış girdilerle, fine-tune edilmiş yöntemlerle yarışabilen BLEU skorları alınmıştır. Bu, görevden bağımsız (task agnostic) yapı fikrini doğurur.
  - Hatalar sistematik değildir: zor bazı görevler iyi giderken daha kolaylarında başarısızlık olabilir. Hocanın GPT-4 örneği: 40 basamaklı iki sayıyı toplayabiliyor, birini 35 basamaklı yapınca toplayamıyor.
- Çoktan seçmeli sorular (ekrandan, 00:50:00)
  - Her seçeneğin, sorunun devamı olarak üretilme olasılığı hesaplanır ve en yüksek olasılıklı seçenek cevap kabul edilir. Modelden “cevap şudur” diye serbest üretim beklenmez.
  - Örnekler: CommonsenseQA (“What blocks sunshine?” {summer | park | desktop | sea | moon}), COPA, ARC. Soru “minimal” ya da “manual” (“The question is: … The answer is: …”) biçimde verilebilir.
- Etiketler yanlış olsa da (ekrandan, 00:52:10)
  - Few-shot örneklerinde duygu etiketleri (Positive, Neutral, Negative) rastgele karıştırılsa bile modelin yeni cümleye doğru cevap verme oranı çok düşmüyor.
  - Etiketler bambaşka anlamsız kelimelerle değiştirilince ise performans gidiyor.
  - Hiç görmediği kelimeler: “Burringo”, “Gigamuru”, “screeg” gibi uydurma kelimeler tanımlanıp cümlede kullanılması istenince model mantıklı cümleler kurabiliyor.
  - Bunun nasıl mümkün olduğuna dair net bir açıklama yok; Bayesçi bir bakış açısıyla açıklama girişimi var.
- OpenAI API (ekrandan, 00:57:40)
  - API’nin yapısı “Completion”: verilen metnin devamını tamamla. Aynı yapıyla çok farklı görevler, farklı görevlermiş gibi çözülebiliyor (örnek: şirketleri kategorilere ayırma; Facebook → Social Media, Fedex → Delivery).
  - GPT-3 makalesi fine-tuning’i gelecek çalışmaya (future work) bıraktı. Bu gelecek çalışma bugün gördüğümüz fine-tune edilmiş dil modelleridir.
- Görevden bağımsız yapı ve RLHF (ekrandan, 01:10:00)
  - Fikir: farklı görevlerin ve veri kümelerinin hepsini insan diliyle ifade edilen ortak bir yapıda eğitmek. İnsan dili sınırsız sayıda görev tanımlama gücü verir (model yeterliyse).
  - Fine-tune, self-supervised (next token) yapıdan insan geri bildirimiyle pekiştirmeli öğrenmeye (RLHF) dönüştürülür.
- Hizalama (alignment) (ekrandan, 01:12:40)
  - Modelin istediğimizi bizim kabul edebileceğimiz bir yolla yapması. “Tüm dünya bilgisine sahip bir insan gibi cevap ver ama zararlı çıktı üretme” amacını matematiksel bir amaç fonksiyonu olarak yazmak zordur, ama örneklerini vermek mümkündür.
  - “Biz kimiz? Kime göre zararlı?” soruları açıktır.
- InstructGPT ve RLHF adımları (ekrandan, 01:20:40)
  - InstructGPT diyalog modeli değildir: bir girdi, bir cevap.
  - 1) Model 1: girdi-çıktı ile eğiticili eğitim (GPT-3’ün supervised fine-tuning’i).
  - 2) Veri kümesi A: bir girdi için Model 1’in farklı çıktıları arasında insanların tercihleri (karşılaştırma verisi).
  - 3) Model 2 (ödül modeli, reward model): veri kümesi A üzerinde eğitilir, insanların

ödül verme mekanizmasını modeller.
  - 4) Model 1, Model 2'nin ödül tahminleriyle RL ile eğitilir; 2. adıma dönülür.
  - 1. ve 3. adımlarda insan devrededir: ilkinde cevap yazar, sonrakinde seçer. Toplamda 20 bin saat insan etkileşimi.
  - Ödül modelinin amacı ölçeklenebilirlik: insanı döngüye sokmak ölçeklenmez; insanın değerlendirmesini taklit eden bir model kurulursa uzun süren 4. adım insansız yapılabilir.
  - İstem veri kümesi (prompt dataset): GPT-3 API'si ücretli açılınca insanların modelden doğru bilgiyi almak için yazdığı istemler. Bunlar çok değerlidir; iş bir istem mühendisliği (prompt engineering) problemine dönüşür.
- Pekiştirmeli öğrenme (RL) temelleri (ekrandan, 01:25:30)
  - Eğiticili öğrenmede "şuna karşı şunu üret" denir, eğiticisizde çıktı yoktur. RL'de doğru cevap söylenmez; yapılanın ne kadar iyi ya da kötü olduğu geri bildirilir.
  - Neden: doğrudan biz de emin olmayabiliriz; tek bir doğru olmayabilir.
  - Ajan çevreyi başta bilmez; hareket eder, her hareketten sonra ödül, ceza ya da nötr bir sinyal alır; amacı cezalardan kaçınıp en çok ödülü toplamaktır.
  - Kullan ya da keşfet ikilemi: bildiğin en iyi yoldan devam etmek ya da daha iyisini bulmak için yeni şeyler denemek. Örnek: öğle yemeğini bildiğimiz en iyi yerde mi, yeni bir yerde mi yiyelim? Genelde keşif olasılığı başta yüksektir, zamanla azaltılır.
  - Ders sonrası not: Slaytta ve anlatımda ikilem "Expedition vs. Exploration" diye geçiyor; doğru terim büyük olasılıkla "exploitation vs. exploration"dır.
- On-policy, off-policy, offline RL (ekrandan, 01:30:00; 01:32:45)
  - Ajan durumu (s_t) gözler, policy $\pi_\theta(a \mid s)$ ile hareket (a) seçer, ödül (r) alır ve sonraki duruma (s_{t+1}) geçer. Policy durumu harekete dönüştüren fonksiyondur; amaç onu optimize etmektir.
  - Dil modeli karşılığı: durum o anki temsil vektörü, policy dil modelinin kendisi (θ parametreleri), hareket üretilen token. Robotta 4 hareket (yukarı, aşağı, sağ, sol) varsa dil modelinde örneğin 32 bin hareket vardır.
  - Toplanan deneyimlerin biçimi $\langle s, a, s', r\rangle$; policy bu deneyimlerle eğitilir. Yöntemlerin farkı deneyimlerin üretim sürecidir.
  - Online RL: o anki policy ile üretilen verilerle güncelleme ($\pi_k \rightarrow \pi_{k+1}$). InstructGPT'de kullanılan budur.
  - Off-policy RL: deneyimler bir tampon (buffer $\mathcal{D}$) içinde birikir; önceki policy sürümlerinin ürettiği veriler de güncellemede kullanılır. Tampon süreci daha kararlı yapar; son adımlardaki kötü bir güncellemenin modeli bozmasını azaltır.
  - Offline RL: modelin farklı sürümlerinden üretilmiş büyük bir veri kümesiyle yeni bir model tamamen çevrimdışı eğitilir.
- Dil modellerini RL ile fine-tune etmek: PPO (ekrandan, 01:40:00)
  - RL'de gradyanlar çok gürültülüdür (stokastik gradyan inişinde örneklerin gradyanları başta ilişkili olduğu halde RL'de bu ilişki zayıftır). Gürültüye karşı temel yaklaşım momentumdur.
  - PPO da buna benzer: model güncellenirken eski halinin çıktılarından çok uzaklaşmak istenmez. Kaybın iki parçası vardır: ödül modelinin verdiği $r_\theta(y \mid x)$ ve KL ıraksaması cezası $-\lambda_{KL} D_{KL}(\pi_{PPO}(y \mid x) \,\|\, \pi_{base}(y \mid x))$.
  - Güncelleme: $\theta \leftarrow \theta + \nabla_\theta J(\theta)$.
  - Model yalnız istem veri kümesine özelleşirse dilin doğallığını (kalitesini) büyük ölçüde kaybediyor; bu yüzden bir önceki sürümden çok uzaklaşılmaz.
  - Hoca: bu alandaki her seçimin arkasında çok teori olmasa bile çok deney vardır.
- Veri kümeleri ve topluluğun tercihleri (ekrandan, 01:42:45)
  - Slayttaki veri kümeleri: Reddit "Explain Like I'm 5", BIG-bench, FLAN, Collective human preferences.
  - İnsan geri bildirimini ucuza elde etmek için forumlardan hazır veri toplanır: bir soruya verilmiş birçok cevaptan hangisinin daha iyi olduğu beğeni/oy sayılarından otomatik çıkarılır (soru, cevap1, cevap2, hangisi iyi). Bu, veri kümesi A'nın karşılığıdır.
  - Stanford human preferences dataset (SHP): zamanı da kullanır; daha çok oy alan ve daha eski olan daha iyi sayılır, çünkü yeni cevap henüz yeterince görülmemiş olabilir.
  - Avantaj: ek maliyet yok. Dezavantaj: oylar her zaman "daha iyi"yi göstermez; özellikle sosyal bilimler konularında beğeni başka nedenlerle verilebilir.
- Açık kaynak çalışmalar: Alpaca (ekrandan, 01:47:30)

  - Meta'nın yayınladığı LLaMA gibi açık modelleri sohbet yeteneği olan görevden bağımsız yapılara dönüştürme çalışmaları hızla artıyor; literatürü takip etmek bile zorlaştı.
  - Alpaca: GPT-3.5 (text-davinci-003, >175B) ile self-instruct yöntemiyle 175 tohum görevden 52 bin talimat-izleme örneği üretilip LLaMA 7B bunlarla eğiticili fine-tune edildi; maliyet 600 dolardan az, ek insan eforu yok.
  - Hoca 52 bin örneği incelediğini, çok çeşitli (diverse) olduğunu söyledi: "10+10=20, 15+15=30" gibi sayıyı şişirmek için üretilmiş tekrarlar değil.
- ChatGPT ve OpenAssistant (ekrandan, 01:50:00; 01:51:30)
  - ChatGPT'nin yöntemi tam bilinmiyor. Bilinenler: InstructGPT'nin bir sürümü; diyalog tabanlı (yeni cevap önceki soru ve cevaplara da bağlı); daha büyük model, daha çok veri.
  - OpenAssistant açık kaynak bir yeniden üretimdir; gönüllülerden veri toplar. Tek girditek çıktı yapısını diyaloğa çevirmek için konuşma ağacı (conversation tree) kullanır.
  - Görevler: konuşmaya cevap yazma (bir sonraki girişi önerme), metni etiketleme (küfür, alay, şiddet içeriyor mu; yararlı, çok kısa, çok belirsiz mi), cevapları kaliteye göre sıralama.
- Henüz yapamadıkları: plan yapma (ekrandan, 01:53:20)
  - Kaynak: Microsoft'un GPT-4 değerlendirmesi ("Sparks of Artificial General Intelligence").
  - Tüm işlem tek bir çizgi üzerinde, soldan sağa ilerler; her şey aynı zorluk seviyesindedir. Çıkmaz sokağa girince başa dönme gibi dinamik bir ileri-geri gidiş yoktur.
  - Aşma girişimleri: Chain of Thought, Tree of Thoughts gibi yaklaşımlar. Hoca, asıl büyük yapının next token prediction ile eğitilmiş kısımda durduğunu, onu değiştirmeden üst katmanlarla oynayarak bunu çözmenin kolay görünmediğini söyledi.
  - Slayttaki sorular: Nasıl aşılabilir? Aşmalı mıyız? Aşılabiliyorsa durdurabilir miyiz?
- Tartışma - 2023 (ekrandan, 02:00:00)
  - Hinton: insan beyni kadar iyi çalışan sistemler yapmaya çalıştık; şimdi 1 trilyon parametreli modeller 100 trilyon parametreli beynimize çok yakın işler yapıyor, yani bilgiyi bizden daha iyi temsil ediyorlar. 100 trilyon: yaklaşık 100 milyar nöron × nöron başına ortalama 1000 bağlantı.
  - Karşı görüş: plan yapamıyorlarsa nasıl kod yazıyorlar? Yepyeni problemleri çözümü olanlar kadar iyi çözemiyorlar. Chomsky bunlara "high-tech plagiarism" diyor: süper kopyalayıcılar. Benzetme menzilleri kısa; çok farklı iki şeyi birleştirip yepyeni bir şey üretemiyorlar. İnsanların soracağı soruların kümesi çok büyük olmadığı için insanları tatmin edebiliyorlar.
  - Benzer iddialar IBM'in Jeopardy sistemi için de söylenmişti ve tutmamıştı: abartmayalım.
  - "İnsanları tahmin edemiyorsan, insanları tahmin edilebilir hale getir." Bu tür araçlar insanların tahmin edilebilirliğini artırır mı?

Hocanın Özellikle Vurguladığı Kısımlar

- Sıfır hata imkânsızlığı ve temsillerin yakınlaşması
  - Dilde aynı bağlamdan sonra birçok farklı devam olduğu için büyük derlemde sıfır hata mümkün değildir. Hatayı azaltmak için model bu farklı devamların temsillerini birbirine yaklaştırır; anlamsal temsillerin kaynağı budur.
- Dil modeli bir sınıflandırıcıdır
  - Her adımda sözlükteki token sayısı kadar sınıftan biri softmax ile seçilir.
- X-shot'ta eğitim yoktur
  - Zero-, one- ve few-shot'ta hiçbir gradyan güncellemesi yapılmaz; "veri kümesi" girdinin içinde, havada oluşturulur.
- Sistematik olmayan hatalar
  - Zor bir işi yapıp kolayını yapamamak (40 basamaklı toplamayı yapıp 35 basamaklıyı yapamamak) insan öğrenmesiyle bağdaşmaz. Hoca bunu "6 basamaklı sayıları çarpabiliyorum ama 3 basamaklıları çarpamıyorum" diyen bir insana benzetti.
- Ödül modeli ölçeklenebilirlik içindir
  - İnsanı döngüden çıkarmak, self-supervised yapıya geçişin de temel motivasyonudur.
- PPO'daki KL cezası
  - Model, istemlere özelleşirken önceki sürümünden çok uzaklaşmamalıdır; yoksa dilin

doğallığı kaybolur.
- Hazır veri (forum oyları) iyi fikirdir ama her zaman istediğimizi ifade etmeyebilir.

Kısa Tekrar Notları

- LLM eğitimi: next / masked / changed token tahmini; günümüzde otoregresif (next thing prediction) standart.
- Büyük derlemde sıfır hata yok: aynı girdi, farklı çıktılar → temsiller yakınlaşır.
- LLM = softmax ile sözlük boyutunda sınıflandırma.
- Emergent properties: fine-tune olmadan yeni görevde sonuç (GPT-3, "Language Models are Few-Shot Learners").
- Zero-shot: görev tanımı + istem; one-shot: + 1 örnek; few-shot: + birkaç örnek; hepsinde gradyan güncellemesi yok.
- LAMBADA: son kelime tahmini; parametre arttıkça üç ayar da iyileşir; few-shot (K=15) en iyi.
- İnsanların makine metnini ayırt etmesi: %75 → %50.
- Çoktan seçmeli: her seçeneğin olasılığı, en büyüğü seçilir.
- Few-shot etiketleri karıştırılsa da başarı çok düşmez; anlamsız etiketlerle düşer.
- OpenAI API = Completion (devamını tamamla).
- Alignment: istediğimizi kabul edilebilir yolla yapmak; "kime göre zararlı?"
- InstructGPT: SFT → karşılaştırma verisi → ödül modeli → RL (PPO) → tekrar; 20 bin saat insan etkileşimi.
- RL: doğru cevap değil iyi/kötü geri bildirimi; kullan ya da keşfet ikilemi; $\langle s, a, s', r \rangle$.
- Online / off-policy (buffer) / offline RL.
- PPO kaybı: $r_\theta(y \mid x) - \lambda_{KL} D_{KL}(\pi_{PPO} \| \pi_{base})$.
- SHP: forum oyları + zaman ile tercih verisi.
- Alpaca: text-davinci-003 + self-instruct (175 tohum → 52K) → LLaMA 7B, <600$.
- ChatGPT: InstructGPT sürümü + diyalog; OpenAssistant: konuşma ağacı, cevap yaz / etiketle / sırala.
- Henüz: plan yapma yok; tek çizgide soldan sağa işlem; Chain of Thought, Tree of Thoughts.
- Hinton (1T vs 100T) ve Chomsky (high-tech plagiarism) görüşleri.

Detaylı Açıklamalar

- **Neden next token prediction?** Ön eğitimin amacı bir problemi çözmek değil, başka problemleri çözmek için sağlam bir altyapı kurmaktır. Her türlü görev "bu metnin devamı ne?" biçiminde ifade edilebildiği için otoregresif modeller yeni görevlere çok daha kolay uyarlanır. Bu yüzden bugün maskeli modelle başlayan firmalar bile sonraki token tahminine geçmiştir.
- **Sıfır hata ve anlam.** Eğitim kümesinde "LLM'ler eğitim kümesinde" ile başlayan binlerce cümle farklı biçimlerde devam eder. Model her biri için doğru devamı üretemez; bu, aynı girdinin farklı sınıflara ait olduğu bir sınıflandırma problemidir. Kaybı en aza indirmenin yolu, aynı bağlamda gelebilen farklı tokenların iç temsillerini benzer noktalara indirgemektir. Hocanın sorusu, benzer anlamlı ya da benzer kullanımlı kelimelerin temsillerinin bu yüzden benzeştiğidir.
- **X-shot öğrenme.** Fine-tune'da yeni görevin örnekleriyle eğitim sürer ve ağırlıklar değişir. X-shot'ta ise görev tanımı ve örnekler girdinin içine yazılır, model yalnızca devamı tahmin eder. Hoca bunu "havada bir veri kümesi oluşturmak" diye anlattı. GPT-3 deneylerinde parametre sayısı arttıkça zero-, one- ve few-shot başarıları birlikte arttı; aritmetik, çeviri ve kelime karıştırma görevlerinde de benzer eğriler görüldü. İnsanların modelin yazdığı haberi ayırt etme doğruluğu büyük modellerde şans düzeyine indi. Öte yandan hataların sistematik olmaması, modellerin insanlar gibi hiyerarşik bir kabiliyet kümesi öğrenmediğini düşündürür.
- **İlginç bulgular.** Few-shot örneklerindeki etiketler rastgele karıştırıldığında performans çok düşmez. Hoca bunu, modele "bunu dersen bunu de" diye açıkça söylenmemesiyle ilişkilendirdi: açık söyleme yalnız ön eğitim ve fine-tune'da olur. Etiketler anlamsız kelimelerle değiştirilince ise performans düşer. Model hiç görmediği, tanımı verilen kelimeleri

de cümlede anlamlı biçimde kullanabilir. Bunun nasıl mümkün olduğunun net bir açıklaması yoktur; Bayesçi bir açıklama girişimi vardır.

- **RLHF süreci.** Önce GPT-3 insanların yazdığı girdi-çıktı örnekleriyle eğiticili olarak fine-tune edilir (Model 1). Sonra Model 1'in aynı girdiye ürettiği farklı çıktılar insanlara sıralatılır (veri kümesi A). Bu tercihlerden insanların değerlendirmesini taklit eden bir ödül modeli (Model 2) öğrenilir. Son olarak yeni istemler için Model 1'in ürettiği cevaplar ödül modeline verilir; gelen ödülle PPO algoritması modelin ağırlıklarını günceller. Güncellenen model yeniden çıktı üretir, yeni tercihler toplanır, ödül modeli yenilenir; 2-4 arası adımlar döngü halinde sürer.
- **RL kavramları ve dil modelleri.** RL oyunlar ve robotlar için geliştirilmiştir; dil modeline uyarlarken durum o anki temsil vektörü, hareket üretilen token, policy dil modelinin kendisidir. Deneyimler $\langle s, a, s', r \rangle$ dörtlüleridir. Online RL'de o anki modelin ürettiği deneyimlerle güncelleme yapılır; off-policy'de deneyimler bir tamponda biriktirilir ve önceki sürümlerin deneyimleri de kullanılır; offline RL'de çevreyle hiç etkileşim olmadan hazır bir veri kümesiyle eğitim yapılır.
- **PPO ve KL cezası.** RL'deki gradyanlar SGD'ye göre çok daha gürültülüdür. Momentum önceki güncellemeyi hesaba katarak gürültüyü azaltır; PPO da benzer biçimde modelin eski çıktılarından çok uzaklaşmamasını sağlar. Ödüle eklenen $-\lambda_{KL} D_{KL}(\pi_{PPO}(y \mid x) \,\|\, \pi_{base}(y \mid x))$ terimi, ayarlanan modelin temel modelden saptığında ceza verir. Böylece model hem kullanıcının isteklerine cevap vermeyi öğrenir hem de dilin doğallığını korur.
- **Tartışma.** Hinton'a göre modeller bilgiyi bizden daha az parametreyle daha iyi temsil ediyor ve bu zekânın bir ölçüsü; kontrol etmek çok zor olabilir. Karşı tarafta Chomsky'nin "high-tech plagiarism" eleştirisi var: modeller benzetme menzili kısa olan süper kopyalayıcılardır. Hocanın yorumu, modellerin plan yapamadığı halde kod yazabilmesinin bir muamma olduğu ve bunun bizimkinden farklı, hiyerarşik olmayan bir öğrenme türüne işaret ettiğidir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.